BSc (Hons) in Information Technology Specializing Data Science

Research Project - IT4010 MLOPs Report

Group ID: 25-26J-088

Project Title: AI-Based Early Childhood Emotional Intelligence Monitoring Platform in Urban areas

1. IT 2231520: Emotion identification
2. IT 22358066: Cognitive memory recall identification
3. IT 22640048: Fine motor & focus level identification
4. IT 22345400: English Language Skills Identification

		IT 2231520	IT 22358066	IT 22640048	IT 22345400
Data Pipeline	**Data Sources:**	Data was collected by visiting preschools in urban areas. Age 4-6 children were taken for this study. 600 artwork and 300 emotion regulation checklists were collected. All the data were validated by a psychologist/art therapist.	The primary method of data collection involves obtaining handwritten letter samples from two different sources, depending on the language. For English handwriting recognition, the dataset was prepared using the EMNIST dataset. The dataset includes uppercase and lowercase characters. For Sinhala handwriting, a custom dataset was developed by collecting handwritten samples in a classroom environment for	Digitized coloring and tracing worksheets from preschool children aged 4–6; 179 coloring samples and 62 tracing samples captured via mobile camera	The system uses two primary datasets. A custom speech dataset was created using recordings from 63 Sri Lankan children aged 4–6 years. A total of 501 WAV files recorded at 16 kHz paired with respective audio-transcription samples were collected with an overall duration of 3.18 hours. The utterances were taken from age-appropriate English

			four Sinhala characters (ඤ, ඦ, ඨ, ඬ). The data was collected from preschool children aged 4-6 in preschools located in the western region of Sri Lanka. The dataset contains images of selected Sinhala letters written by students under real learning conditions, resulting in a total of 3075 handwritten letter samples. Each child contributed multiple samples under supervised classroom conditions.		storybooks and recorded in real classroom environments to naturally include classroom background noise and realistic accent variations. A custom text dataset of 6300 rows was created to reflect common grammatical mistakes. This dataset was specifically designed to simulate real spoken grammar mistakes rather than adult-written formal text.
	Data Preprocessing:	The artwork were preprocessed using techniques such as resizing, converting image to tensor, normalizing the pixel to adapt with the imagenet. Using the standardscaler for the 1024 features to have a mean of 0 and 1.	The handwriting images were preprocessed using separate pipelines for English and Sinhala because both scripts have different structural complexity. For the general English handwriting pipeline, the input image was first loaded and converted into	Images decoded from raw byte streams, resized to 500×500px (INTER_AREA), converted BGR→RGB; HSV color space used for segmentation	Audio preprocessing was performed using the Hugging Face datasets library. The steps included are loading all WAV files, Resampling audio to 16 kHz and Standardizing audio input for Whisper model compatibility. For the transcription

		Reducing the features to 150 using principal component analysis. For the erc dataset the target variable was ordinal encoder to be converted to numerical values.	grayscale. Contrast normalization was then applied to improve faint pencil strokes, followed by background detection to identify whether the image had a light or dark background. Gaussian blur was used for denoising, and adaptive thresholding was applied to convert the image into a binary format with white letters on a black background. Since many preschool handwriting samples were written on ruled or lined paper, the system also included ruled-line detection and removal. This step handled both solid and dotted notebook lines. After that, morphological closing was applied to connect fragmented strokes, which is important because young children often write letters with broken or incomplete strokes. Finally, the image was resized and normalized		texts, Converted text to lowercase, removed unnecessary punctuation and special characters,preserved essential punctuation and removed extra whitespace. Importantly,no grammar correction no auto-correction No sentence restructuring was done. This ensured the model learned from authentic child speech patterns rather than artificially cleaned text. The dataset was split using random seed 42 into: training set of 80% (400 samples), validation set 10% (50 samples), test set 10% (51 samples). The grammar dataset was formatted using the text-to-text learning approach

			to a standard size while maintaining the aspect ratio. For Sinhala handwriting, a separate preprocessing pipeline was implemented. Sinhala images were converted to grayscale, very small images were upscaled, and high-contrast or already-binary images were handled by checking image polarity. Gaussian blur and adaptive thresholding were applied, followed by ink-ratio normalization to reduce overly thick strokes. Morphological opening and closing were used to remove small noise and connect gaps. The main letter region was then cropped using contour detection, centered, and resized with padding to 128×128 pixels. This higher resolution was used to preserve the curved and		required by Flan-T5. Each record was transformed as Input: Correct this: {ungrammatical sentence} , Output: ErrorType: {error category}, Correction: {corrected sentence}. Additional preprocessing included label consistency checking, error type distribution visualization, stratified sampling based on ErrorType to maintain balanced class distributionDataset split was done as a training set of 75%, validation set of 12.5%. test set of 12.5%. This ensured stable training and reliable evaluation across all grammar categories.

			complex stroke patterns of Sinhala letters.		
	Data storage and versioning	Images were kept in the google drive, for easier connection with the colab interface.	The handwriting datasets were stored locally in organized project folders according to language and dataset type. English handwriting data was maintained separately from the custom Sinhala handwriting dataset, making it easier to manage training, testing, and evaluation files. Preprocessed datasets were also saved after cleaning and normalization to ensure consistent input data during model training and evaluation. Golden reference letters were stored separately and kept unchanged to maintain fair and consistent structural comparison. Model versioning was handled by saving trained models as separate	Images were stored locally on the laptop in research dataset folder including coloring and tracing folder. The dataset was a fixed snapshot 179 coloring and 62 tracing images collected once and kept unchanged for the entire study.	All datasets were stored in Google Drive for easy model training with colab. Version control was maintained to ensure consistency across experiments. For ASR training, TensorBoard was used to track validation WER, loss curves and training progress across epochs. Model checkpoints were saved during training using Hugging Face Seq2SeqTrainer, allowing rollback to the best-performing validation state through early stopping.Fixed random seed was used across all experiments to ensure

			checkpoint files without overwriting previous versions. Training history files were also stored in JSON format to track accuracy, loss, and performance changes over time.		reproducibility of results.
Model Development	**Model Selection**	The artwork was trained on multiple models. Deep learning models were mainly used for the training of the data due to the limited number of data in the 3 categories. The best model for the artwork was selected, by evaluating metrics such as accuracy, confusion, roc curve, classification report. For artwork explainability the model was chosen by analyzing the accuracy and the speed of the explainability.	Initially, multiple deep learning models were tested for handwritten letter recognition, including Siamese Networks, VGG16, and Convolutional Neural Networks (CNNs). These models were evaluated based on accuracy, computational efficiency, and suitability for handwritten character recognition. For the English letter recognition task, a Modified LeNet-5 CNN architecture was selected as it provided the best balance between performance and efficiency when working with the EMNIST dataset (52 classes).	Initially a deep learning model was evaluated for image-based assessment. However, due to the limited dataset size (179 coloring, 62 tracing samples) the model achieved insufficient accuracy. A rule-based computer vision approach was therefore selected, prioritising transparency and interpretability for educators. For coloring, four metrics are computed via OpenCV: Completeness, Boundary Adherence, Neatness, and	For speech-to-text transcription, Whisper-small.en was selected as the base model. This transformer-based encoder-decoder model was chosen because it performs strongly on noisy speech, supports robust transcription under accented English conditions, and is suitable for fine-tuning on smaller custom datasets. Since the target users are young Sri Lankan children speaking English as a second language, Whisper provided

		The erc data was trained on machine learning model, due to simplicity of the data. The model selection for the data was taken considering metric such as accuracy, confusion matrix and classification report.	It performed better than heavier models like VGG16 while maintaining lower computational cost. For the Sinhala letter recognition task, a CNN architecture based on LeNet was also selected, but with additional convolutional layers (SinhalaCNN) to better capture the complex curves and structures of Sinhala characters. Since Sinhala letters are more visually complex than English letters, a deeper architecture provided better feature extraction and higher accuracy.	Consistency. For tracing: Completeness, Line Accuracy, Neatness, and Consistency. Scores are aggregated using a configurable Weighted Summation formula with age-stratified thresholds, achieving Cronbach's Alpha of 0.8664 (coloring) and 0.7900 (tracing).	better generalization than traditional ASR approaches. The small.en version was selected instead of larger Whisper variants to balance transcription accuracy with training efficiency and deployment feasibility. For grammar assessment, Flan-T5-base was selected using the text-to-text generation paradigm. This model was chosen because it can simultaneously perform grammatical error classification and corrected sentence generation within a single inference step. For pragmatic evaluation, Sentence-BERT (SBERT) using the

					all-MiniLM-L6-v2 model was selected because it provides fast sentence embedding generation with strong semantic understanding while maintaining low inference cost. For tense and temporal alignment detection, spaCy (en_core_web_sm) was selected because model performs POS tagging, dependency parsing efficiently for real-time classroom deployment. For session difficulty personalization, Q-Learning was selected instead of deep reinforcement learning methods because the problem contains are limited

					state space and limited action space.
	Model Training	Three methods were used to train the artwork data. Firstly a deep learning methods, such as restnet, but this method could not learn the dataset and predict results accurately with increased epochs, learning rate and batch size. Secondly a feature engineering method was utilized. This method gave better results than the deep learning method. The feature engineering happened using features that were found in previous research papers. Features included, saturation, color theory, edge density. The third approach involved a hybrid model. This will extract the features	For the English model, training was carried out using the EMNIST dataset. The dataset was split into training, validation, and testing sets. Data augmentation techniques such as rotation, scaling, translation, brightness variation, and noise addition were applied to improve generalization. The model was trained using the AdamW optimizer with a cosine annealing learning rate scheduler. Early stopping was used to prevent overfitting. During training, the model achieved a best validation accuracy of 86.56%, and the final test accuracy was 86.73% with a test loss of 0.346. These results show that the model performs well across different handwriting styles.	No traditional model training was conducted as the framework employs a rule-based computer vision pipeline. Assessment parameters including metric weights and age-stratified thresholds are externally configurable by educators through the human-in-the-loop interface, allowing teachers to adjust scoring sensitivity based on professional expertise or individual child needs. Default weight distributions are provided as a baseline (e.g. Completeness, Boundary Adherence, Neatness, Consistency for coloring; Completeness, Line	ASR model training was performed progressively rather than directly fine-tuning the full model. The training stages included, Fully frozen encoder, Partial encoder unfreezing (last 6 layers), Full encoder-decoder fine-tuning. This progressive strategy helped evaluate how much encoder adaptation was necessary for Sri Lankan child speech recognition. Two augmentation strategies were tested as applied once before training and pplied dynamically during each epoch. Training was executed using

		from a vit_l_16 model and then pass it to a svm model to give the prediction. This model was able to get a training accuracy of 92.73% and a testing accuracy of 85.18%. Two methods were used to experiment with the explainability. Lime model was able to identify the regions of the feature in the model, but it took a long time to give the results. So instead a heatmap method was used to visualize the features that the model found. The erc dataset is a simple rating dataset of the child's behaviour. Therefore simple machine learning models were used for this. Knn model was	For the Sinhala model the dataset was split into training (80%), validation (10%), and testing (10%) sets. Data augmentation techniques such as rotation, scaling, noise addition, and stroke variation were applied. The Sinhala model was trained using the AdamW optimizer with a cosine annealing learning rate scheduler and early stopping. The model achieved a best validation accuracy of 97.7%, and the final test accuracy was 98.71% with a test loss of 0.043. This high accuracy indicates strong performance in recognizing Sinhala characters. Overall, both models were evaluated using training accuracy, validation accuracy, and test accuracy, and the best-performing models were selected based on validation results.	Accuracy, Neatness, Consistency for tracing) but are not fixed values. Internal consistency of the default configuration was validated using Cronbach's Alpha, achieving 0.8664 for the coloring module (n=179) and 0.7900 for the tracing module (n=62), confirming the reliability of the pipeline under standard settings.	Hugging Face Seq2SeqTrainer on NVIDIA GPU hardware. Hyperpatrameters ware tuned by changing Optimizer, learning rate, scheduler, effective Batch Size: 32, mixed precision, label smoothing, early Stopping, gradient accumulation enabled and gradient checkpointing enabled. Performance was evaluated using validation WER and test WER. The Flan-T5-base model was trained using supervised fine-tuning on the custom grammar dataset. Each training sample followed the text-to-text format. This enabled the model to jointly predict both

		able to generate an accuray of training 84% and testing 81%. Decision tree model was overfitted to the dataset. Since the data were linear separable a svm model was used, which gave the best results. This was able to produce an accuracy of training 95% and testing 93%.			grammar error type and corrected output. Performance evaluation used Error type accuracy, correction accuracy and the overall accuracy. The Q-learning agent was trained using a simulated environment containing 1,000 virtual student profiles across 200 episodes, representing approximately 18,000 learning interactions. The agent used an epsilon-greedy strategy for exploration and exploitation.
	Tools used	The model was written in python language. It was then exported to the python backend environment. The frontend is running using the react js. The	The model was developed using Python and integrated into the backend environment. The frontend was developed using React JS, while the backend was implemented using Flask.	Python serves as the primary language for the assessment engine, leveraging OpenCV for image preprocessing, template extraction, and metric	The Hugging Face Transformers library was used as the core framework for both the ASR and Grammar Error Detection

Model Integration		backend and the frontend is connected using the flask. The models are exported using the pickle files. The user will enter the details into the react frontend, through REST APIs the data will be send to the backend where the models are, after the models are generated the results will be generated for the user.	The frontend and backend are connected through REST APIs. The trained models were exported using pickle files and loaded in the backend for inference. Users upload handwritten inputs through the React frontend, and the data is sent to the backend where preprocessing and model prediction are performed. The results are then generated and returned to the user. Python serves as the main programming language for the assessment engine. OpenCV and NumPy were used for image preprocessing and segmentation, while PyTorch was used for CNN model inference. Flask was used to handle REST API communication, process image uploads, and return results in JSON format. React was used to develop	computation. Flask provides the REST API backend, handling image uploads and returning JSON assessment results. Python was also used for frontend-backend communication. React was employed for the educator dashboard and human-in-the-loop configuration interface. Assessment results and child metadata are stored persistently in MongoDB. IBM SPSS Statistics was used to validate internal consistency of the scoring pipeline via Cronbach's Alpha analysis.	modules. It was used for fine-tuning the Whisper-small.en model for speech recognition, fine-tuning the Flan-T5-base model for grammar error detection and correction, tokenization and sequence preparation, model checkpoint saving and training and evaluation pipeline management. The Gemini LLM API was used to generate age-appropriate conversational prompts for children. The entire system pipeline was implemented using Python Flask for backend and React for frontend. The datasets library was used for efficient loading and preprocessing of the custom ASR dataset. PyTorch used as the

			the user interface and dashboard components. Gemini LLM was integrated to generate personalized feedback. MongoDB was used to store uploaded input images along with the generated results for future reference and analysis.		backend deep learning framework for all transformer model training. The sentence transformers library was used to implement semantic relevance detection using SBERT. spaCy with the en_core_web_sm model was used for pragmatic tense analysis and time mismatch detection. TensorBoard was used for experiment monitoring and training visualization. Training experiments were executed using Colab.
Model Deployment	**Testing Environments**	The models were tested in the colab environment, due to the ability to use the GPUs in the environment. All the data were uploaded to the driver to easily	The system was tested locally using real classroom handwriting samples. Users uploaded images via the React interface, and outputs such as segmentation results, scores, and feedback were verified visually.	The system was tested on a local laptop environment using the collected research dataset of 179 coloring samples and 62 tracing samples. Each image was uploaded through the React dashboard,	The developed models were tested primarily in the Google Colab environment due to its access to high-performance NVIDIA GPUs.

		connect with the environment. The data was validated by psychologists to ensure the results that are generated are corrected.		which visually displayed the detected template boundaries and highlighted regions influencing the generated scores, enabling manual verification of the computer vision pipeline's detection accuracy. Statistical validation was conducted using IBM SPSS Statistics to measure internal consistency via Cronbach's Alpha, achieving 0.8664 for the coloring module and 0.7900 for the tracing module, confirming the reliability of the scoring pipeline across the full dataset.	To ensure data quality and reliability of outputs, the collected child language data and generated results were validated with expert review to confirm that grammar corrections, linguistic assessments, and feedback remained educationally appropriate for young learners.
	Deployment Platform	Docker is the platform in which the system is integrated. Through the localhost we can view the system	The system was deployed using Docker containers, enabling consistent execution across environments. The	Local machine via Docker container; React frontend accessed through browser on localhost	The system was deployed using Docker containers for easy execution across different development

			application was accessed via a web browser on localhost.		environments. Able to access via a web browser on localhost.
	Deployment Method	Docker images are created for each component, to help with library conflicts.	Dockerized deployment; Flask backend and React frontend containerized and run via Docker	Dockerized deployment; Flask backend and React frontend containerized and run via Docker	Dockerized deployment. Flask backend and React frontend containerized and run via Docker
Future Enhancements	**Model Improvement:**	Currently the artwork dataset was labeled only for three emotions, happy, angry, sad. Future enhancements would be to increase the emotion identification capabilities.	Expand Sinhala dataset to include full character set for better generalization; Explore additional deep learning architectures to improve recognition accuracy and scoring reliability.	Expand tracing dataset to strengthen reliability estimates; integrate deep learning segmentation for improved detection accuracy; add longitudinal tracking to monitor developmental progress over time	One key improvement is upgrading the Automatic Speech Recognition module by exploring larger Whisper variants or parameter-efficient fine-tuning methods such as LoRA. Future work can focus on expanding the training dataset with more diverse child speech patterns.